

Short overview of SQL (in Oracle DBMS) for Computer Science BSc students

Made by Vágner Anikó

2018

This work was supported by the construction EFOP-3.4.3-16-2016-00021.
The project was supported by the European Union, co-financed by the European Social Fund.

1 Introduction

SQL - Structured Query Language Types of SQL Statements:

- DDL - Data Definition Language: CREATE, ALTER, DROP
- DML - Data Manipulation Language: INSERT, UPDATE, DELETE
- QL - Query Language (it can be a part of DML statements): SELECT
- DCL - Data Control Language:
 - Handling privileges: GRANT, REVOKE
 - Transaction Control: COMMIT, ROLLBACK, SAVEPOINT

2 CREATE TABLE , ALTER TABLE and DROP TABLE statements

2.1 CREATE TABLE statement

Use the CREATE TABLE statement to create a relational table.

```
create table keepers  
(k_id number(5),  
keeper_name varchar2(50) not null,
```

```

birth_date date,
identity_card_number varchar2(50),
constraint keeper_pk primary key (k_id),
constraint keeper_cn_uq unique (identity_card_number));

create table pets
(pet_name varchar2(50),
keeper_id number(5),
birth_date date,
constraint p_pk primary key (pet_name, keeper_id, birth_date),
constraint p_k_fk foreign key (keeper_id) references keepers(k_id));

```

In a basic level you can use the following data types: DATE, CHAR, VARCHAR2 and NUMBER.

- VARCHAR2(size): Variable-length character string having maximum length size characters. You must specify size for VARCHAR2.
- NUMBER [(p [, s])]: Number having precision p and scale s. p is the precision, or the maximum number of significant decimal digits. s is the scale, or the number of digits from the decimal point to the least significant digit.
 - NUMBER(5): for integer with 5 digits
 - NUMBER(5,2): You can store there for example 999.99 or 123.45 values
 - NUMBER(5,-2): You can store there for example 9999900 or 1234500 values
 - NUMBER: for floating-point numbers
- CHAR[(size)]: Fixed-length character data of length size characters.
 - CHAR: 1 character
 - CHAR(5): string with 5 characters. If you want to store there 'nut' it will store 'nut ' (with 2 spaces at the end)

2.2 ALTER TABLE statement

Use the ALTER TABLE statement to alter the definition of a relational table.

Example 1 *Add a new column to a table:*

```
alter table keepers
add (address varchar2(50));
```

Example 2 *Modify a column of a table:*

```
alter table keepers
modify (address varchar2(100));
```

Example 3 *Rename a column of a table:*

```
alter table keepers
rename column address to ad;
```

Example 4 *Eliminate a column from a table:*

```
alter table keepers
drop column ad;
```

Example 5 *Eliminate a constraint from a table:*

```
alter table pets
drop constraint p_pk;
```

Example 6 *Eliminate the primary key from a table:*

```
alter table pets
drop primary key;
```

Example 7 *Add a new constraint to a table:*

```
alter table pets
add constraint p_pk primary key (pet_name, keeper_id, birth_date);
```

Example 8 *Rename a constraint of a table:*

```
alter table pets
rename constraint p_pk to pe_pk;
```

2.3 DROP TABLE statement

Use the DROP TABLE statement to remove the table and all its data from the database entirely.

```
drop table pets;
```

3 DML statements

3.1 INSERT statement

Use the INSERT statement to add rows to a table.

```
insert into keepers (k_id, keeper_name,  
                    birth_date, identity_card_number)  
values (1,'John_Smith',  
        to_date('1990.01.01','yyyy.mm.dd'), null);
```

```
insert into keepers (k_id, keeper_name, birth_date)  
values (2,'Jane_Smith',to_date('1993.05.01','yyyy.mm.dd'));
```

```
insert into pets (pet_name, keeper_id, birth_date)  
values ('Sammy', 1, to_date('2000.01.01','yyyy.mm.dd'));
```

```
insert into pets (pet_name, keeper_id, birth_date)  
values ('Cooper', 1, to_date('2000.01.01','yyyy.mm.dd'));
```

You can insert multiple rows into a table if you use the insert into statement with a subquery.

```
insert into pets (pet_name, keeper_id, birth_date)  
select 'John_Ball', 1000, sysdate  
from dual;
```

3.2 UPDATE statement

Use the UPDATE statement to change existing values in a table.

```
update keepers  
set identity_card_number='12345'  
where k_id=1;
```

```
update keepers  
set keeper_name='Jonatan_Apple',  
    birth_date =to_date('2000.01.01', 'yyyy.mm.dd')  
where k_id=1;
```

```

update pets
set keeper_id=(select k_id from keepers
               where keeper_name='Jane_Smith')
where keeper_id=(select k_id from keepers
                 where keeper_name='John_Smith');

```

3.3 DELETE statement

Use the DELETE statement to remove rows from a table.

```

delete pets
where keeper_id=1;

```

Example 9 *Remove all rows from the pets table*

```

delete pets;

```

4 SELECT statement

In this part HR schema is used.

Use a SELECT statement or subquery to retrieve data from one or more tables.

Example 10 *Retrieve all rows from a table:*

```

select *
from hr.employees;

```

4.1 WHERE clause

The WHERE condition lets you restrict the rows selected to those that satisfy one or more conditions.

Example 11 *List the employees whose first name is David:*

```

select *
from hr.employees
where first_name='David';

```

Example 12 *List the employees whose name is David Austin:*

```
select *
from hr.employees
where first_name='David' and last_name='Austin';
```

Example 13 *List the employees whose first name is David or whose salary is more than 1000:*

```
select *
from hr.employees
where first_name='David' or salary>1000;
```

Example 14 *List the employees whose name is David Austin or whose salary is more than 1000:*

```
select *
from hr.employees
where first_name='David' and last_name='Austin' or salary>1000;
```

Example 15 *List the employees whose name is David Austin or Steven Markle and whose salary is more than 1000:*

```
select *
from hr.employees
where (first_name='David' and last_name='Austin' or
      first_name='Steven' and last_name='Markle') and salary>1000;
```

Example 16 *List the employees who have no manager:*

```
select *
from hr.employees
where manager_id is null;
```

Example 17 *List the employees who have manager:*

```
select *
from hr.employees
where manager_id is not null;
```

Example 18 *List the name and the employee_id of each employee whose first name begins with S and its third letter is e:*

```
select first_name, last_name, employee_id
from hr.employees
where first_name like 'S_e\%';
```

Example 19 *List the name, employee_id and department_id of each employee who works on the department which id is 30, 60 or 70:*

```
select first_name, last_name, employee_id, department_id
from hr.employees
where department_id in (30,60,70);
```

Example 20 *List the name, the employee_id and the department_id of each employee who works outside the department which id is 30, 60 or 70:*

```
select first_name, last_name, employee_id, department_id
from hr.employees
where department_id not in (30,60,70);
```

Example 21 *List the employees whose first name are Bruce and whose salary are between 1000 and 20000:*

```
select first_name, last_name, employee_id, department_id
from hr.employees
where salary between 1000 and 20000 and first_name='Bruce'
```

4.2 ORDER BY clause

Use the ORDER BY clause to specify the order in which cells on the left-hand side of the rule are to be evaluated.

You can specify an ascending (asc or nothing) or a descending (desc) order. You can specify whether returned rows containing null values should appear first or last in the ordering sequence with NULLS FIRST or NULLS LAST clause.

```
select first_name, last_name, employee_id, department_id
from hr.employees
where department_id not in (30,60,70)
order by department_id desc, last_name, first_name;
```

```
select first_name, last_name, employee_id, department_id, salary
from hr.employees
where department_id not in (30,60,70)
order by salary;
```

```

select *
from hr.departments
order by manager_id nulls first;

select *
from hr.departments
order by manager_id desc nulls last;

```

4.3 Important single-row functions and concatenation operator

- ABS: for absolute value
- ROUND: to round numbers
- TRUNC: to truncate numbers

```

SELECT ABS(-15),
ROUND(15.63), ROUND(15.63,1), ROUND(15.63,-1),
TRUNC(15.63), TRUNC(15.63,1), TRUNC(15.63,-1)
FROM DUAL;

```

- CONCAT: to concat strings
- || operator: to concat strings

```

SELECT CONCAT('apple','tree'), 'apple' || 'tree' FROM DUAL;

```

- LOWER: to produce lowercase letters
- UPPER: to produce uppercase letters

```

SELECT LOWER(last_name), UPPER(last_name) FROM employees;

```

- SUBSTR: The SUBSTR(char, character_position, substring_length) functions return a portion of char, beginning at character_position, substring_length characters long.

```

SELECT SUBSTR('ABCDEFGF',3,4) FROM DUAL;

```

- LENGTH: It returns the length of a string.

```

SELECT LENGTH('ABCDEFGF') FROM DUAL;

```

- INSTR: The INSTR(string, substring) functions search string for substring.


```
SELECT INSTR('CORPORATE_FLOOR','OR', 3, 2) FROM DUAL;
```

- SYSDATE: returns the current date and time set for the operating system on which the database server resides.

- ADD_MONTHS: returns the date date plus some months given in the parameter.

```
SELECT ADD_MONTHS(sysdate, -3) FROM DUAL;
```

- MONTHS_BETWEEN: returns number of months between dates date1 and date2.

```
SELECT MONTHS_BETWEEN
(TO_DATE('2018.01.01','yyyy.mm.dd'),
TO_DATE('2010.03.30','yyyy.mm.dd'))
FROM DUAL;
```

- TO_DATE: converts char to a value of DATE data type.
- TO_CHAR (with DATE data type parameter): converts a date to VARCHAR2 type.

```
SELECT to_date('2018.01.01','yyyy.mm.dd'),
to_char(sysdate,'yyyy.mm.dd_hh24:mi:ss')
FROM DUAL;
```

- NVL: lets you replace null (returned as a blank) with another value

```
SELECT commission_pct, nvl(commission_pct, 0)
FROM HR.EMPLOYEES;
```

- DECODE

```
SELECT product_id,warehouse_id,
DECODE (warehouse_id, 1, 'Southlake',
                2, 'San_Francisco',
                3, 'New_Jersey',
                4, 'Seattle',
                'Non_domestic') "Location"
FROM oe.inventories;
```

- USER: returns the name of the session user (the user who logged on).

```
SELECT USER FROM DUAL;
```

4.4 GROUP BY and HAVING clauses

Specify the GROUP BY clause if you want the database to group the selected rows based on the value of an expression or expressions for each row and return a single row of summary information for each group.

If you also specify a group by clause in a select statement, then the select list can contain only the following types of expressions:

- Aggregate functions (count, sum, avg, min, max)
- Expressions identical to those in the group by clause
- Constants
- The functions USER, UID, and SYSDATE
- Expressions involving the preceding expressions that evaluate to the same value for all rows in a group

Example 22 *List the sum, average, minimum and maximum of the salaries of the employees, moreover count the number of employees for each jobs (job_id is enough):*

```
select job_id, count(*), sum(salary),  
       avg(salary), min(salary), max(salary)  
from hr.employees  
group by job_id;
```

Example 23 *List the number of managers (of employees) for each jobs (job_id is enough):*

```
select job_id, count(manager_id)  
from hr.employees  
group by job_id;
```

Example 24 *List the number of different managers (of employees) for each jobs (job_id is enough):*

```
select job_id, count(distinct manager_id)  
from hr.employees  
group by job_id;
```

Example 25 *List the number of employees whose salary is more than 10000 for each jobs (job_id is enough). Order the list by the number of employees:*

```

select job_id, count(employee_id) co
from hr.employees
where salary>10000
group by job_id
order by co desc

```

Use the HAVING clause to restrict the groups of returned rows to those groups for which the specified condition is TRUE. If you omit this clause, then the database returns summary rows for all groups.

Example 26 *List the sum, average, minimum and maximum of the salaries and count the number of employees for each jobs (job_id is enough) whose average salary is more than 10000:*

```

select job_id, count(*) co, sum(salary) sums,
       avg(salary) avgs,
       min(salary) mins, max(salary) maxs
from hr.employees
group by job_id
having avg(salary)>10000
order by avgs desc;

```

4.5 JOINS

A join is a query that combines rows from two or more tables. A join is performed whenever multiple tables appear in the FROM clause of the query. The select list of the query can select any columns from any of these tables. If any two of these tables have a column name in common, then you must qualify all references to these columns throughout the query with table names to avoid ambiguity.

Most join queries contain at least one join condition, either in the FROM clause or in the WHERE clause. The join condition compares two columns, each from a different table. To execute a join, the DBMS combines pairs of rows, each containing one row from each table, for which the join condition evaluates to TRUE. The columns in the join conditions need not also appear in the select list.

If two tables in a join query have no join condition, then the DBMS returns their **Cartesian product**, which means that the DBMS combines each row of one table with each row of the other. A Cartesian product always generates many rows and is rarely useful.

Example 27 *Cartesian product*

```
select *
from hr.employees, hr.jobs;
```

An **inner join** (sometimes called a simple join) is a join of two or more tables that returns only those rows that satisfy the join condition.

Example 28 *Inner join*

```
select *
from hr.employees e inner join
     hr.departments d on e.department_id=d.department_id
```

Example 29 *Inner join*

```
select *
from hr.employees e, hr.jobs j
where e.job_id=j.job_id
```

Example 30 *List the name of each employee along with the name of the employee's manager:*

```
select e1.last_name||' works for '||e2.last_name
from employees e1, employees e2
where e1.manager_id = e2.employee_id
order by e1.last_name;
```

An **outer join** extends the result of a simple join. An outer join returns all rows that satisfy the join condition and also returns some or all of those rows from one table for which no rows from the other satisfy the join condition. To write a query that performs an outer join of tables A and B and returns all rows from A (a **left outer join**), use the LEFT [OUTER] JOIN syntax in the FROM clause. For all rows in A that have no matching rows in B, the DBMS returns null for any select list expressions containing columns of B.

Example 31 *Left outer join*

```
select *
from hr.employees e left outer join
     hr.departments d
on e.department_id=d.department_id;
```

To write a query that performs an outer join of tables A and B and returns all rows from B (a **right outer join**), use the RIGHT [OUTER] JOIN syntax in the FROM clause. For all rows in B that have no matching rows in A, the DBMS returns null for any select list expressions containing columns of A.

Example 32 *Right outer join*

```
select *
from hr.employees e
     right outer join hr.departments d
     on e.department_id=d.department_id;
```

To write a query that performs an outer join and returns all rows from A and B, extended with nulls if they do not satisfy the join condition (a **full outer join**), use the FULL [OUTER] JOIN syntax in the FROM clause.

Example 33 *Full outer join*

```
select *
from hr.employees e full outer join
     hr.departments d
     on e.department_id=d.department_id;
```

4.6 Subqueries

A subquery answers multiple-part questions. For example, to determine who works in Lorentz's department, you can first use a subquery to determine the department in which Lorentz works. You can then answer the original question with the parent SELECT statement. A subquery in the FROM clause of a SELECT statement is also called an inline view. A subquery in the WHERE clause of a SELECT statement is also called a nested subquery.

Example 34 *List each employee who works in Lorentz's department*

```
select last_name, department_id
from employees
where department_id =
     (select department_id
      from employees
      where last_name = 'Lorentz')
order by last_name, department_id;
```

If columns in a subquery have the same name as columns in the containing statement, then you must prefix any reference to the column of the table from the containing statement with the table name or alias. To make your statements easier to read, always qualify the columns in a subquery with the name or alias of the table.

Example 35 *List each employee who works in Lorentz's department*

```

select e.last_name, e.department_id
from hr.employees e, (select department_id
                      from hr.employees
                      where last_name = 'Lorentz') d
where e.department_id = d.department_id
order by e.last_name, e.department_id;

```

A subquery can contain another subquery.

Example 36 *List the employees whose departments are located in Toronto city:*

```

select first_name, last_name
from hr.employees
where department_id in
  (select department_id
   from hr.departments
   where location_id in
     (select location_id
      from hr.locations
      where city='Toronto'));

```

The DBMS performs a **correlated subquery** when a nested subquery references a column from a table referred to a parent statement one level above the subquery. The parent statement can be a SELECT, UPDATE, or DELETE statement in which the subquery is nested. A correlated subquery conceptually is evaluated once for each row processed by the parent statement.

A correlated subquery answers a multiple-part question whose answer depends on the value in each row processed by the parent statement. For example, you can use a correlated subquery to determine which employees earn more than the average salaries for their departments. In this case, the correlated subquery specifically computes the average salary for each department.

Example 37 *List the employees who earn more than the average salaries for their departments*

```

select department_id, last_name, salary
  from employees x
 where salary > (select avg(salary)
                 from employees
                 where x.department_id = department_id)
 order by department_id;

```

4.7 The Set Operators

You can combine multiple queries using the set operators UNION, UNION ALL, INTERSECT, and MINUS.

The corresponding expressions in the select lists of the component queries of a compound query must match in number and must be in the same data type group (such as numeric or character).

If the select list preceding the set operator contains an expression, then you must provide a column alias for the expression in order to refer to it.

- UNION: it returns only distinct rows that appear in either result. It eliminates duplicate selected rows.

```
SELECT location_id, department_name "Department",  
       TO_CHAR(NULL) "Warehouse" FROM departments  
UNION  
SELECT location_id, TO_CHAR(NULL) "Department", warehouse_name  
FROM warehouses;
```

```
SELECT product_id FROM order_items  
UNION  
SELECT product_id FROM inventories;
```

- UNION ALL: it returns all rows. It does not eliminate duplicate selected rows.

```
SELECT location_id FROM locations  
UNION ALL  
SELECT location_id FROM departments;
```

- INTERSECT: it returns only those unique rows returned by both queries.

```
SELECT product_id FROM inventories  
INTERSECT  
SELECT product_id FROM order_items;
```

- MINUS: it returns only unique rows returned by the first query but not by the second.

```
SELECT product_id FROM inventories  
MINUS  
SELECT product_id FROM order_items;
```

4.8 Using ROWNUM in Top-n Reporting

Example 38 *List the employees with the 10 smallest salaries.*

```
SELECT *
FROM (SELECT * FROM employees ORDER BY salary)
WHERE ROWNUM < 11;
```

4.9 SELECT examples

Example 39 *List all employees' names with their department name. The result has to include the employees who have no departments.*

```
select first_name, last_name, d.department_name
from hr.employees e left outer join
    hr.departments d
    on e.department_id=d.department_id;
```

Example 40 *How many employees belong to each department? List the departments' names which have some employees. Order the list by the number of employees descending.*

```
select count(employee_id) co, d.department_name
from hr.employees e inner join
    hr.departments d
    on e.department_id=d.department_id
group by d.department_name
order by co desc;
```

Example 41 *How many employees belong to each department? List all the departments' names even if no employees work there. Order the list by the number of employees descending.*

```
select count(employee_id) co, d.department_name
from hr.employees e right outer join
    hr.departments d
    on e.department_id=d.department_id
group by d.department_name
order by co desc;
```

Example 42 *How many employees belong to each city? List all the cities even if no employees work there. Order the list by the number of employees descending.*


```

select count(employee_id) co, l.city
from hr.employees e right outer join
    hr.departments d
    on e.department_id=d.department_id
    right outer join
    hr.locations l
    on d.location_id=l.location_id
group by l.city
order by co desc;

```

Example 43 *Which jobs are done by more than 3 employees? How many employees do each of them? Order the list by the number of employees descending.*

```

select j.job_title, count(employee_id) co
from hr.employees e inner join
    hr.jobs j
    on e.job_id=j.job_id
group by j.job_id, j.job_title
having count(employee_id)>3
order by co desc

```

Example 44 *Which departments have less than 3 employees? How many employees belong to each of them? Order the list by the number of employees descending.*

```

select d.department_name, count(employee_id) co
from hr.employees e right outer join
    hr.departments d
    on e.department_id=d.department_id
group by d.department_id, d.department_name
having count(employee_id)<3
order by co desc;

```

Example 45 *Which are the departments whose employees' total salary is more than 10000? Order the list by the total salary descending.*

```

select d.department_name, sum(e.salary) s
from hr.employees e right outer join
    hr.departments d
    on e.department_id=d.department_id
group by d.department_id, d.department_name
having sum(salary)>10000
order by s desc;

```

Example 46 *Which are the departments whose employees' total salary is less than 10000 (even 0, because it has no employee)? Order the list by the total salary descending.*

```
select d.department_name, sum(nvl(e.salary,0)) s
from hr.employees e right outer join
      hr.departments d
      on e.department_id=d.department_id
group by d.department_id, d.department_name
having sum(nvl(salary,0))<10000
order by s desc;
```

Example 47 *Who is the employee whose salary is the highest?*

```
select *
from hr.employees
where salary=(select max(salary) from hr.employees);
```

Example 48 *On which department does the employee work whose salary is the highest?*

```
select department_name
from hr.departments
where department_id in (select department_id
                        from hr.employees
                        where salary=(select max(salary) from hr.employees));
```

Example 49 *Which job is done by the most employees?*

```
select job_title
from hr.jobs
where job_id in (select job_id
                  from hr.employees
                  group by job_id
                  having count(employee_id)=(select max(count(employee_id))
                                              from hr.employees
                                              group by job_id))
```

Example 50 *Which department has the fewest employees (even 0)?*

```

select department_name, count(employee_id)
from hr.departments d left outer join hr.employees e
    on d.department_id=e.department_id
group by d.department_id, department_name
having count(employee_id)=(select min(count(employee_id))
                           from hr.departments d left outer join hr.employees e
                               on d.department_id=e.department_id
                           group by d.department_id, department_name)

```

Example 51 *Which employee has the highest salary in each department?*

```

select d.department_name, nn.*
from hr.departments d inner join
    (select e.department_id, e.employee_id,
         e.first_name, e.last_name
     from hr.employees e inner join
         (select department_id, max(salary) s
          from hr.employees
          group by department_id) n
       on e.department_id=n.department_id
       and e.salary=n.s) nn
on d.department_id=nn.department_id;

```

Example 52 *Which departments have no employees?*

```

select d.*
from hr.departments d
where department_id not in
    (select e.department_id
     from hr.employees e);

```

5 Other important DDL statements

5.1 CREATE VIEW statement

Use the CREATE VIEW statement to define a view, which is a logical table based on one or more tables or views. A view contains no data itself.

```

SELECT k_id, keeper_name, count(keeper_id) number_of_pets
FROM keepers ke left outer join pets pe on ke.k_id=pe.keeper_id
GROUP BY k_id, keeper_name

```

5.2 CREATE SEQUENCE statement

Use the CREATE SEQUENCE statement to create a sequence, which is a database object from which multiple users may generate unique integers. You can use sequences to automatically generate primary key values.

```
CREATE SEQUENCE seq1;
select seq1.nextval from dual;
select seq1.currval from dual;
insert into keepers (k_id, keeper_name,
birth_date, identity_card_number)
values (seq1.nextval, 'John_Smith',
to_date('1990.01.01', 'yyyy.mm.dd'), null);
```

5.3 CREATE SYNONYM statement

Use the CREATE SYNONYM statement to create a synonym, which is an alternative name for a table, view, sequence, another synonym or other database objects.

```
CREATE SYNONYM syn_keepers for keepers;
select * from syn_keepers;
```

6 DCL statements

6.1 Transactions

A transaction is a logical, atomic unit of work that contains one or more SQL statements.

A transaction groups SQL statements so that they are either all committed, which means they are applied to the database, or all rolled back, which means they are undone from the database.

A transaction consists of one of the following:

- One or more data manipulation language (DML) statements that together constitute an atomic change to the database
- One data definition language (DDL) statement (it can be transaction control statement, too)

6.2 COMMIT statement

Use the COMMIT statement to end your current transaction and make permanent all changes performed in the transaction. A transaction is a sequence

of SQL statements that Oracle Database treats as a single unit. This statement also erases all savepoints in the transaction and releases transaction locks.

Until you commit a transaction:

- You can see any changes you have made during the transaction by querying the modified tables, but other users cannot see the changes. After you commit the transaction, the changes are visible to other users' statements that execute after the commit.
- You can roll back (undo) any changes made during the transaction with the ROLLBACK statement.

Oracle Database issues an implicit COMMIT under the following circumstances:

- Before any syntactically valid data definition language (DDL) statement, even if the statement results in an error
- After any data definition language (DDL) statement that completes without an error

Oracle recommends that you explicitly end every transaction in your application programs with a COMMIT or ROLLBACK statement, including the last transaction, before disconnecting from Oracle Database. If you do not explicitly commit the transaction and the program terminates abnormally, then the last uncommitted transaction is automatically rolled back.

A normal exit from most Oracle utilities and tools causes the current transaction to be committed. A normal exit from an Oracle precompiler program does not commit the transaction and relies on Oracle Database to roll back the current transaction.

```
COMMIT;
```

6.3 ROLLBACK statement

Use the ROLLBACK statement to undo work done in the current transaction

```
ROLLBACK;
```

6.4 SAVEPOINT statement

Use the SAVEPOINT statement to create a name for a system change number (SCN), to which you can later roll back.

```
UPDATE employees
SET salary = 7000
WHERE last_name = 'Banda';

SAVEPOINT banda_sal;

UPDATE employees
SET salary = 12000
WHERE last_name = 'Greene';

SAVEPOINT greene_sal;

SELECT SUM(salary) FROM employees;

ROLLBACK TO SAVEPOINT banda_sal;

UPDATE employees
SET salary = 11000
WHERE last_name = 'Greene';

COMMIT;
```

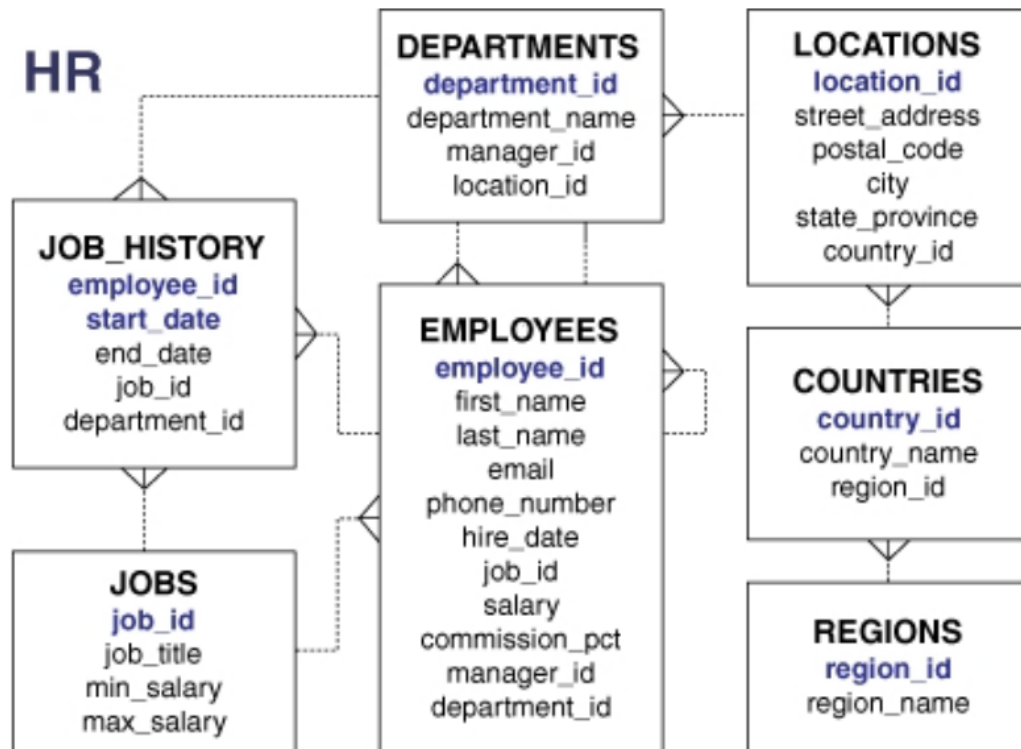
6.5 GRANT statement

Use the GRANT statement to grant privileges to users.

6.6 REVOKE statement

Use the REVOKE statement to Revoke privileges from users.

7 HR schema



8 Literature

- Oracle 12c SQL documentation:
<https://docs.oracle.com/database/121/SQLRF/toc.htm>